

Homework 03 – Oh the Places You'll Go

Authors: Emma, Sreya, Owen, Melanie, Chelsea, Xavier, Eric

Topics: abstract classes, overriding, equals, toString

Problem Description

You are hired by a start-up in Tech Square aiming to deliver the next big transport app. The cars and scooters have been bought – your co-workers are just waiting for the code! And so, you set out to revolutionize transportation.

Solution Description

You will need to complete and turn in 3 classes: Vehicle.java, Car.java, and Bus.java.

Notes:

1. All variables should be inaccessible from other classes, require an instance to be accessed through, **AND** must **NOT** have a getter nor setter, unless specified otherwise.
2. All methods should be accessible from everywhere and must require an instance to be accessed through, unless specified otherwise.
3. **Use constructor chaining and reuse methods whenever possible!** Ensure that your constructor chaining helps you **maximize code reuse!**
4. **Reuse code when possible** and helper methods are optional.
5. Make sure to add Javadoc comments to your methods and classes!

Vehicle.java

This class represents a vehicle to get around the city. Vehicle cannot be instantiated. Instead, it should only serve as a superclass for more specific vehicles.

Variables:

- `String id` – the identifier of this vehicle; IDs should be kept a secret.
 - This field should be **immutable** after construction of an instance.
- `double earnings` – the total amount of money this vehicle has earned transporting passengers
- `int numMiles` – the number of miles this vehicle has travelled
 - This field should be accessible by subclasses of Vehicle.
- `String[] passengers` – the passengers aboard this vehicle; each item in the array represents a passenger's name
 - This field should be accessible by subclasses of Vehicle.

Constructors:

- A constructor that takes in `id`, `numMiles`, `passengers`, and sets `earnings` to 0.0.
- A constructor that takes in `id` and `passengers`, and sets `numMiles` and `earnings` to 0.0.
- Assume all arguments are valid.

THIS GRADED ASSESSMENT IS NOT FOR DISTRIBUTION.
ANY DUPLICATION OUTSIDE OF GEORGIA TECH'S LMS IS UNAUTHORIZED.

Methods:

- `canDrive`
 - Takes in an `int` representing the distance to travel in miles and returns a `boolean` representing whether the vehicle can travel that far.
 - This method will **not** have an implementation in the `Vehicle` class.
 - **Hint:** What keyword allows us to declare a method in a class without providing an implementation?
 - This method should not modify the state of this object.
- `calculateCost`
 - Takes in an `int` representing the distance to travel in miles and returns a `double` representing the cost for this vehicle to travel that far.
 - This method will **not** have an implementation in the `Vehicle` class.
 - **Hint:** What keyword allows us to declare a method in a class without providing an implementation?
 - This method should not modify the state of this object.
- `addPassengers`
 - Takes in an `int` representing the distance to travel and a `String[]` representing new passengers, and returns a `boolean` value.
 - Returns `true` if all new passengers can fit in the vehicle **and** the vehicle can travel the specified distance.
 - Returns `false` if the new passengers cannot fit in the vehicle or the vehicle cannot travel the given distance.
 - This method will **not** have an implementation in the `Vehicle` class.
 - **Hint:** What keyword in the method header allows us to accomplish this?
- `chargeRide`
 - Takes in an `int` representing the distance to travel and returns nothing.
 - Increases `numMiles` and `earnings` accordingly.
 - **Note:** You should only charge the ride if this vehicle can travel the specified distance.
- `chargeRide`
 - Takes in an `int` representing the distance to travel and an `int` representing the number of passengers to charge, and returns nothing.
 - Increases `numMiles` and `earnings` accordingly.
 - The amount earned should be the total cost for each passenger to ride.
 - **Note:** You should only charge the ride if this vehicle can travel the specified distance.
- `equals`
 - Overrides the `equals` method from `Object`.
 - Two vehicles should be considered equal if they have equal `id` and `numMiles`.
- `toString`
 - Overrides the `toString` method from `Object`.
 - Returns a `String` in the following format (single line, without leading/trailing whitespace, angle brackets, and quotation marks):

```
"<id> has travelled <numMiles> miles and has earned <earnings> dollars."
```
 - **Note:** Floating point values should be displayed to two decimal places with rounding.
- **Reuse code if possible.**

Car.java

This class represents a regular car in the transportation network and is a concrete subtype of `Vehicle`.

Variables:

- `double rate` – the cost of using this car to travel one mile
- `double fees` – the fee charge to the ride to use this car once
- `int maxNumMiles` – the maximum number of miles that this car can travel before it is retired

Constructors:

- A constructor that takes in `id`, `numMiles`, `passengers`, `rate`, `fees`, and `maxNumMiles`, and sets `earnings` to 0.0.
- A constructor that takes in `id`, `numMiles`, and `maxNumMiles`, and sets `earnings` to 0.0, `passengers` to an empty array of length 4, `rate` to 10.0, and `fees` to 15.0.
- A constructor that takes in `id`, and sets `numMiles` to 0, `passengers` to an empty string array of length 4, `rate` to 10.0, `fees` to 15.0, and `maxNumMiles` to 200.
- Assume all arguments are valid.

Methods:

- `canDrive`
 - Takes in an `int` representing the distance to travel in miles and returns a `boolean` representing whether this car can travel the specified distance without exceeding the maximum number of miles.
 - **Note:** A car cannot drive if the specified distance is negative.
 - Be sure to consider how far the car has already travelled.
 - This method should not modify the state of this object.
- `calculateCost`
 - Takes in an `int` representing the distance to travel in miles and returns a `double` representing the cost for this car to travel that far.
 - You may assume we will not test this method with a negative distance.
 - Returns -1.0 if this car cannot travel the specified distance without exceeding its maximum number of miles or if the specified distance is negative.
 - This method should not modify the state of this object.
- `addPassengers`
 - Takes in an `int` representing the distance to travel and a `String[]` representing new passengers.
 - Each string in the array represents a passenger's name. You may assume this array will not contain any null elements, and it contains at least one element.
 - Returns a `boolean` value.
 - Each passenger should be added to the array represented by the `passengers` field, starting from the lowest available index.
 - **Note:** Passengers should only be added if there are seats available for **ALL** passengers in this car **AND** this car can travel the specified distance.
 - Indices are available if they are not already occupied by a passenger. However, do not assume that passenger names will always be at the front of the array. For example, a valid passenger array given to the constructor could be `[null, "Melanie", "Chelsea", null]`.

THIS GRADED ASSESSMENT IS NOT FOR DISTRIBUTION.
ANY DUPLICATION OUTSIDE OF GEORGIA TECH'S LMS IS UNAUTHORIZED.

- If all new passengers fit in this car, charge the ride assuming the entire group is going together. That is, no matter how many new passengers there are, `earnings` and `numMiles` should only be increased once (i.e., the amount earned for this ride is not dependent on the number of new passengers).
 - **Hint:** Reuse code! What method can you use to accomplish this?
- Returns `true` if this car can travel the specified distance **and** all new passengers can fit in this car, and `false` otherwise.
- `equals`
 - Overrides the `equals` method from `Vehicle`.
 - Two cars should be considered equal if they have equal `id`, `numMiles`, `rate`, `fees`, and `maxNumMiles`.
- `toString`
 - Overrides the `toString` method from `Vehicle`.
 - Returns a `String` in the following format (single line, without leading/trailing whitespace, angle brackets, and quotation marks):

```
"Car <id> has travelled <numMiles> miles and has earned
<earnings> dollars. It can only drive <maxNumMiles>
miles. It costs <rate> dollars per mile and there is a
one-time fee of <fees> dollars."
```
 - Floating point values should be displayed to two decimal places with rounding.
- **Reuse code if possible.**

Bus.java

This class represents a regular bus in the transport network, such as a Georgia Tech bus or a MARTA bus and is a concrete subtype of `Vehicle`.

Variables:

- `String location` – the general area this bus operates in
 - For example, it could be “Georgia Tech”, “Midtown”, “Ponce”, or something else.
- `int stopsPerMile` – the number of stops this bus makes in one mile

Constructors:

- A constructor that takes in `id`, `numMiles`, `location`, `stopsPerMile`, and sets `passengers` to an empty string array of length 20.
- A constructor that takes in the `id`, `location`, and sets `numMiles` to 0, `stopsPerMile` to 2, and `passengers` to an empty array of length 20.
- Assume all arguments are valid.

Methods:

- `canDrive`
 - Takes in an `int` representing the distance to travel and returns a `boolean` representing whether this bus can travel that far.
 - Since busses are well kept, they will always be ready to drive, unless the distance is negative.
 - This method should not modify the state of this object.

THIS GRADED ASSESSMENT IS NOT FOR DISTRIBUTION.
ANY DUPLICATION OUTSIDE OF GEORGIA TECH'S LMS IS UNAUTHORIZED.

- `calculateCost`
 - Takes in an `int` representing the distance to travel and returns the `double` cost for the bus to drive that far, per passenger.
 - The price is calculated by taking the specified distance multiplied by 3 and then dividing by `stopsPerMiles`. In other words, the less stops a bus makes the more it'll cost to ride!
 - **Hint:** Be wary of data types while performing operations!
 - Returns `-1.0` if this bus cannot drive.
 - This method should not modify the state of this object.
- `addPassengers`
 - Takes in an `int` representing the distance to travel and a `String[]` representing new passengers.
 - Each string in the array represents a passenger's name. You may assume this array will not contain any null elements, and it contains at least one element.
 - Returns a `boolean` value.
 - Each passenger should be added to the array represented by the `passengers` field, starting from the lowest available index.
 - **Note:** Passengers should only be added if this bus can travel the specified distance.
 - Passengers should be added to the array represented by the `passengers` field in order, starting from the front of the argument array.
 - If there are passengers that cannot fit on the bus, do nothing with them.
 - Indices are available if they are not already occupied by a passenger. However, do not assume that passenger names will always be at the front of the array. For example, a valid passenger array given to the constructor could be `[null, "Melanie", "Chelsea", null]`.
 - **Note:** Not all passengers from the array argument need to fit on the bus for any passengers to be added.
 - Each passenger that boards the bus should be charged for the ride – it does not matter if they are riding together with anyone else.
 - **Hint:** What method can you use to charge multiple passengers?
 - Returns `true` if this bus can travel the given distance and `false` otherwise, no matter the number of new passengers that boarded the bus.
- `equals`
 - Overrides the `equals` method from `Vehicle`.
 - Two busses should be considered equal if they have equal `id`, `numMiles`, `location`, and `stopsPerMile`.
- `toString`
 - Overrides the `toString` method from `Vehicle`.
 - Returns a `String` in the following format (single line, without leading/trailing whitespace, angle brackets, and quotation marks):

```
"Bus <id> has travelled <numMiles> miles and has earned <earnings> dollars. This bus drives around <location> and makes <stopsPerMile> stops per mile."
```
- **Reuse code if possible.**

Driver.java

This class will be used to test your code.

Methods:

- `main`
 - Create two `Car` objects.
 - Use `addPassengers(int, String[])` on at least one of the `Car` objects.
 - Call `toString()` on both `Car` objects.
 - Check to see if the two `Car` objects are equal.
 - Create two `Bus` objects.
 - Use `addPassengers(int, String[])` on at least one of the `Bus` objects.
 - Call `toString()` on both `Bus` objects.
 - Check to see if the 2 `Bus` objects are equal.
 - These tests and the ones on Gradescope are by no means comprehensive, so be sure to create your own!

Checkstyle

You must run Checkstyle on your submission (to learn more about Checkstyle, check out [cs1331-style-guide.pdf](#) under the Checkstyle Resources module on Canvas). **The Checkstyle cap for this assignment is 20 points.** This means there is a maximum point deduction of 20. If you don't have Checkstyle yet, download it from Canvas → Modules → Checkstyle Resources → `checkstyle-8.28.jar`. Place it in the same folder as the files you want to run Checkstyle on. Run Checkstyle on your code like so:

```
$ java -jar checkstyle-8.28.jar YourFileName.java
Starting audit...
Audit done.
```

The message above means there were no Checkstyle errors. If you had any errors, they would show up above this message, and the number at the end would be the number of points we would take off (limited by the Checkstyle cap). In future assignments we will be increasing this cap, so get into the habit of fixing these style errors early!

Additionally, you must Javadoc your code.

Run the following to only check your Javadocs:

```
$ java -jar checkstyle-8.28.jar -j yourFileName.java
```

Run the following to check both Javadocs and Checkstyle:

```
$ java -jar checkstyle-8.28.jar -a yourFileName.java
```

For additional help with Checkstyle see the CS 1331 Style Guide.

Turn-In Procedure

Submission

To submit, upload the files listed below to the corresponding assignment on Gradescope:

- `Vehicle.java`
- `Car.java`
- `Bus.java`
- `Driver.java`

Make sure you see the message stating the assignment was submitted successfully. From this point, Gradescope will run a basic autograder on your submission as discussed in the next section. **Any autograder tests are provided as a courtesy to help “sanity check” your work and you may not see all the test cases used to grade your work.** You are responsible for thoroughly testing your submission on your own to ensure you have fulfilled the requirements of this assignment. If you have questions about the requirements given, reach out to a TA or Professor via the class forum for clarification.

You can submit as many times as you want before the deadline, so feel free to resubmit as you make substantial progress on the assignment. We will only grade your latest submission. **Be sure to submit every file each time you resubmit.**

Gradescope Autograder

If an autograder is enabled for this assignment, you may be able to see the results of a few basic test cases on your code. Typically, tests will correspond to a rubric item, and the score returned represents the performance of your code on those rubric items only. If you fail a test, you can look at the output to determine what went wrong and resubmit once you have fixed the issue. **We reserve the right to hide any or all test cases, so you should make sure to test your code thoroughly against the assignment's requirements.**

The Gradescope tests serve two main purposes:

- Prevent upload mistakes (e.g., non-compiling code)
- Provide basic formatting and usage validation

In other words, the test cases on Gradescope are by no means comprehensive. Be sure to thoroughly test your code by considering edge cases and writing your own test files. You also should avoid using Gradescope to compile, run, or Checkstyle your code; you can do that locally on your machine.

Other portions of your assignment can also be graded by a TA once the submission deadline has passed, so the output on Gradescope may not necessarily reflect your grade for the assignment.

Burden of Testing

You are responsible for thoroughly testing your submission against the written requirements to ensure you have fulfilled the requirements of this assignment.

Be **very careful** to note the way in which text output is formatted and spelled. Minor discrepancies could result in failed autograder cases.

If you have questions about the requirements given, reach out to a TA or Professor via the class forum for clarification.

Allowed Imports

To prevent trivialization of the assignment, you may not import any classes for this assignment.

Feature Restrictions

There are a few features and methods in Java that overly simplify the concepts we are trying to teach or break our autograder. For that reason, do not use any of the following in your final submission:

- `var` (the reserved keyword)
- `System.exit`
- `System.arraycopy`

Collaboration

Only discussion of the assignment at a conceptual high level is allowed. You can discuss course concepts and assignments broadly; that is, at a conceptual level to increase your understanding. If you find yourself dropping to a level where specific Java code is being discussed, that is going too far. Those discussions should be reserved for the instructor and TAs. To be clear, you should never exchange code related to an assignment with anyone other than the instructor and TAs.

The only code you may share are test cases written in a Driver class. If you choose to share your Driver class, they should be posted to the assignment discussion thread on the course discussion forum. We encourage you to write test cases and share them with your classmates, but we will not verify their correctness (i.e., use them at your own risk).

You **MAY NOT** use code generation tools to complete this assignment. This includes generative AI tools such as ChatGPT.

Important Notes (Don't Skip)

- Non-compiling files will receive a 0 for all associated rubric items.
- Do not submit `.class` files.
- Test your code in addition to the basic checks on Gradescope.
- Submit every file each time you resubmit.
- Read the "Allowed Imports" and "Restricted Features" to avoid losing points.
- **Check on Ed Discussion for a note containing all official clarifications and sample outputs.**

It is expected that everyone will follow the Student-Faculty Expectations document and the Student Code of Conduct. The professor expects a **positive, respectful, and engaged academic environment** inside the classroom, outside the classroom, in all electronic communications, on all file submissions, and on any document submitted throughout the duration of the course. **No inappropriate language is to be used, and any assignment deemed by the professor to contain inappropriate offensive language or threats will get a zero.** You are to use professionalism in your work. Violations of this conduct policy will be turned over to the Office of Student Integrity for misconduct.